Lab 6: Vending Machine FSM

CMPE 125 · Vivado 2025.2 · October 22, 2026

Implementation

The design uses the button-driven controller in the supplied reference PDF. There is one synthesizable module, `vending_machine`, and one testbench, `vending_machine_tb`. Simulation and the Basys 3 board use that same module. The PDF's state encodings, initialization, and entire clocked block are preserved. The only interface change is keeping `state` internal because the supplied constraints do not assign pins to a state output.

The soda costs 25 cents. Switches select a nickel, dime, or quarter, and a sampled button press accepts the selected coin. `button_prev` stores the previous sampled button value. The condition `button && ! button_prev` recognizes a rising edge. A continuously high button accepts only one coin for clean input signals; releasing it permits another insertion.

The positive clock edge updates credit and the registered dispense/change outputs. Outputs retain their values until the next accepted button edge or reset. A new press first clears the previous outputs, then computes the result for the selected coin and old credit. These are registered outputs; the testbench checks them after the clock edge and after releasing the button.

`reset` is asynchronous and active high. It clears credit, `button_prev`, and all four outputs. The PDF also initializes these registers to zero. Without a button edge, coin switch changes have no effect. A button edge with no selected coin clears outputs and holds credit. With several selected coins, the PDF gives priority to nickel, then dime, then quarter.

State encoding and transitions

The source declares five credit states using three bits:

State	Credit	Encoding
S0	0 cents	000
S1	5 cents	001
S2	10 cents	010
S3	15 cents	011
S4	20 cents	100

Three bits can represent eight codes and are sufficient for five credit states. `button_prev` and the four held outputs require additional storage. Synthesis may optimize or recode the source state representation.

Each cell below gives **next state / output result** for an accepted press with exactly one selected coin. Let `Y = {Dispense, ReturnNickel, ReturnDime, ReturnTwoDimes}`.

Credit state	Nickel	Dime	Quarter
S0	S1 / 0000	S2 / 0000	S0 / 1000
S1	S2 / 0000	S3 / 0000	S0 / 1100
S2	S3 / 0000	S4 / 0000	S0 / 1010
S3	S4 / 0000	S0 / 1000	S0 / 1110
S4	S0 / 1000	S0 / 1100	S0 / 1001

A vending transition clears credit to S0 and records the appropriate result. `1110` means dispense and return a nickel plus a dime (15 cents). `1001` means dispense and return two dimes (20 cents). A subsequent coin belongs to the next purchase. In the PDF's default case, an unused state returns to S0 on an accepted button edge.

Basys 3 connections and demo

The FPGA target is `xc7a35tcpg236-1` with a 100 MHz clock on W5. All ten active pin assignments match the supplied PDF.

Control/output	Board control	Pin
Nickel	SW0	V17
Dime	SW1	V16
Quarter	SW2	W16
reset	SW3	W17
button	BTNC	U18
Dispense	LD3	V19
ReturnNickel	LD2	U19
ReturnDime	LD1	E19
ReturnTwoDimes	LD0	U16

Open with `./lab open lab_6`. Run Behavioral Simulation first, then generate the bitstream and program the board through Hardware Manager. The generated file is `lab_6/build/lab6/lab6.runs/impl_1/vending_machine.bit`. Reset by raising and lowering SW3. Select one coin with SW0, SW1, or SW2, then press and release BTNC. A quarter alone lights LD3. A nickel followed by a quarter lights LD3 and LD2. The seven-segment display is not used.

The reference PDF directly samples asynchronous controls and does not include synchronization or button debounce. Physical contact bounce may count extra coins. The timing constraints exclude asynchronous control input paths and static LED output paths; these exceptions do not add synchronization or debounce. Internal paths remain timed at 100 MHz. Physical board behavior still requires testing after programming.

Verification

The single testbench reproduces both PDF transactions and their timing:

- Nickel followed by quarter: soda plus 5 cents change (`1100`).
- Dime, dime, nickel: soda with no change (`1000`).

Additional checks visit all five credit states and try each coin, covering all 15 single-coin transitions. An independent arithmetic reference computes the next credit and change from cents. All change amounts (0, 5, 10, 15, and 20 cents) are exercised. Tests also check held buttons, held outputs, asynchronous reset, coin switches without a button press, empty selections, and priority for multiple selected coins. Failures stop with `$fatal`.

Run the testbench from the repository root:

```
mkdir -p lab_6/build
(cd lab_6/build && vivado -mode batch -source ../scripts/verify.tcl)
```

The expected successful transcript is:

```
Test 1 PASS: Soda and 5 cents change
Test 2 PASS: Soda and no change
PASS: PDF board FSM: both PDF examples, all 15 coin transitions, held outputs/button, reset, switch isolation, and input
priority.
```

Build and check the implementation with:

```
(cd lab_6/build && vivado -mode batch -source ../scripts/bitstream.tcl)
```

The build writes internal timing, utilization, and DRC reports under `build/`. The bitstream and generated reports are excluded from Git. The appendices contain the exact design source, single testbench, and board constraints.

Appendix A: synthesizable Verilog

```
`timescale 1ns / 1ps

// Copied from the supplied PDF.
// State is internal because the PDF has no pin assignments for a state output.
module vending_machine (
    input wire clk,
    input wire reset,
    input wire button,
    input wire Nickel,
    input wire Dime,
    input wire Quarter,
    output reg Dispense,
    output reg ReturnNickel,
    output reg ReturnDime,
    output reg ReturnTwoDimes
);

    // Five states
    parameter S0 = 3'b000,
              S1 = 3'b001,
              S2 = 3'b010,
              S3 = 3'b011,
              S4 = 3'b100;

    reg [2:0] state;
    reg button_prev;

    initial begin
        state = S0;
        button_prev = 0;
        Dispense = 0;
        ReturnNickel = 0;
        ReturnDime = 0;
        ReturnTwoDimes = 0;
    end
    always @(posedge clk or posedge reset) begin

        if (reset) begin
            state <= S0;
            button_prev <= 0;
            Dispense <= 0;
            ReturnNickel <= 0;
            ReturnDime <= 0;
            ReturnTwoDimes <= 0;
        end

        else begin
            button_prev <= button;

            // Check if the button was just pressed
            if (button && !button_prev) begin

                Dispense <= 0;
                ReturnNickel <= 0;
                ReturnDime <= 0;
                ReturnTwoDimes <= 0;

                case (state)

                    S0: begin
                        if (Nickel)
```

```
            state <= S1;
        else if (Dime)
            state <= S2;
        else if (Quarter) begin
            state <= S0;
            Dispense <= 1;
        end
    end

    S1: begin
        if (Nickel)
            state <= S2;
        else if (Dime)
            state <= S3;
        else if (Quarter) begin
            state <= S0;
            Dispense <= 1;
            ReturnNickel <= 1;
        end
    end

    S2: begin
        if (Nickel)
            state <= S3;
        else if (Dime)
            state <= S4;
        else if (Quarter) begin
            state <= S0;
            Dispense <= 1;
            ReturnDime <= 1;
        end
    end

    S3: begin
        if (Nickel)
            state <= S4;
        else if (Dime) begin
            state <= S0;
            Dispense <= 1;
        end
        else if (Quarter) begin
            state <= S0;
            Dispense <= 1;
            ReturnNickel <= 1;
            ReturnDime <= 1;
        end
    end

    S4: begin
        if (Nickel) begin
            state <= S0;
            Dispense <= 1;
        end
        else if (Dime) begin
            state <= S0;
            Dispense <= 1;
            ReturnNickel <= 1;
        end
        else if (Quarter) begin
            state <= S0;
            Dispense <= 1;
            ReturnTwoDimes <= 1;
        end
    end

    default: state <= S0;
```

```
                endcase
            end
        end
    end

endmodule
```

Appendix B: simulation testbench

```
`timescale 1ns / 1ps

module vending_machine_tb;
    reg clk = 0, reset = 0, button = 0;
    reg Nickel = 0, Dime = 0, Quarter = 0;
    wire Dispense, ReturnNickel, ReturnDime, ReturnTwoDimes;
    wire [3:0] leds = {Dispense, ReturnNickel, ReturnDime, ReturnTwoDimes};
    integer credit_state, coin_index, total, change, next_state;
    reg [3:0] expected;

    vending_machine uut (
        .clk(clk), .reset(reset), .button(button),
        .Nickel(Nickel), .Dime(Dime), .Quarter(Quarter),
        .Dispense(Dispense), .ReturnNickel(ReturnNickel),
        .ReturnDime(ReturnDime), .ReturnTwoDimes(ReturnTwoDimes)
    );
    always #5 clk = ~clk;

    task check;
        input [2:0] state;
        input [3:0] result;
        begin
            if (uut.state !== state || leds !== result)
                $fatal(1, "state=%b LEDs=%b; expected state=%b LEDs=%b",
                       uut.state, leds, state, result);
        end
    endtask

    task clear_machine;
        begin
            @(negedge clk); reset = 1; button = 0;
            {Nickel, Dime, Quarter} = 0;
            #1; check(0, 0); // Asynchronous reset, before the next clock edge.
            @(negedge clk); reset = 0;
        end
    endtask

    task insert;
        input [2:0] coin;
        begin
            @(negedge clk); {Nickel, Dime, Quarter} = coin; button = 1;
            repeat (4) @(posedge clk); // Holding the button must count only once.
            @(negedge clk); {Nickel, Dime, Quarter} = 0; button = 0;
            @(posedge clk); #1; // Check the held result after button release.
        end
    endtask

    initial begin
        #1; check(0, 0);
        // The two transactions and input timing from the PDF's testbench.
        reset = 1;
        #11 reset = 0;
        #10 Nickel = 1; button = 1;
        #10 Nickel = 0; button = 0;
        #10 Quarter = 1; button = 1;
        #10 Quarter = 0; button = 0;
        check(0, 4'b1100);
        $display("Test 1 PASS: Soda and 5 cents change");

        #10 reset = 1;
        #10 reset = 0;
```

```
        #10 Dime = 1; button = 1;
        #10 Dime = 0; button = 0;
        #10 Dime = 1; button = 1;
        #10 Dime = 0; button = 0;
        #10 Nickel = 1; button = 1;
        #10 Nickel = 0; button = 0;
        check(0, 4'b1000);
        $display("Test 2 PASS: Soda and no change");

        // Independently check every credit/coin pair using cents arithmetic.
        for (credit_state = 0; credit_state < 5; credit_state = credit_state + 1) begin
            for (coin_index = 0; coin_index < 3; coin_index = coin_index + 1) begin
                clear_machine();
                repeat (credit_state) insert(3'b100);
                check(credit_state, 0);
                total = credit_state * 5;
                case (coin_index)
                    0: begin total = total + 5;  insert(3'b100); end
                    1: begin total = total + 10; insert(3'b010); end
                    2: begin total = total + 25; insert(3'b001); end
                endcase
                expected = 0;
                next_state = total / 5;
                if (total >= 25) begin
                    change = total - 25;
                    next_state = 0;
                    expected = {1'b1, (change % 10 == 5),
                                (change == 10 || change == 15), (change == 20)};
                end
                check(next_state, expected);
            end
        end
        clear_machine();
        @(negedge clk); Quarter = 1;
        repeat (4) @(posedge clk);
        #1; check(0, 0); // A switch alone does not insert a coin.
        insert(3'b000); check(0, 0);
        // Match the PDF's priority: nickel first, then dime, then quarter.
        insert(3'b111); check(1, 0);
        insert(3'b011); check(3, 0);
        insert(3'b001); check(0, 4'b1110);
        insert(3'b000); check(0, 0); // A new press clears the previous result.
        insert(3'b001); check(0, 4'b1000);
        clear_machine();

        $display("PASS: PDF board FSM: both PDF examples, all 15 coin transitions, held outputs/button, reset, switch
isolation, and input priority.");
        $finish;
    end
    initial begin
        #100000;
        $fatal(1, "Board simulation timeout");
    end
endmodule
```

Appendix C: Basys 3 pin constraints

```
# Basys 3 rev B: matches the vending-machine reference PDF's used pins.
# Pin reference: https://github.com/Digilent/digilent-xdc/blob/master/Basys-3-Master.xdc
# Apply to the vending_machine design top.

set_property -dict { PACKAGE_PIN W5 IOSTANDARD LVCMOS33 } [get_ports clk]
create_clock -name sys_clk_pin -period 10.000 [get_ports clk]

# SW0 = nickel, SW1 = dime, SW2 = quarter, SW3 = active-high reset.
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports Nickel]
set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports Dime]
set_property -dict { PACKAGE_PIN W16 IOSTANDARD LVCMOS33 } [get_ports Quarter]
set_property -dict { PACKAGE_PIN W17 IOSTANDARD LVCMOS33 } [get_ports reset]

# Center pushbutton accepts the selected coin on its sampled rising edge.
set_property -dict { PACKAGE_PIN U18 IOSTANDARD LVCMOS33 } [get_ports button]

# LED0 = two dimes, LED1 = dime, LED2 = nickel, LED3 = dispense.
set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports ReturnTwoDimes]
set_property -dict { PACKAGE_PIN E19 IOSTANDARD LVCMOS33 } [get_ports ReturnDime]
set_property -dict { PACKAGE_PIN U19 IOSTANDARD LVCMOS33 } [get_ports ReturnNickel]
set_property -dict { PACKAGE_PIN V19 IOSTANDARD LVCMOS33 } [get_ports Dispense]

# Human controls have no clock relationship (sampled directly as in the PDF).
# These exceptions do not add synchronization or debounce to the PDF circuit.
set_false_path -from [get_ports {button Nickel Dime Quarter reset}]
# LEDs are static indicators with no external synchronous receiver.
set_false_path -to [get_ports {Dispense ReturnNickel ReturnDime ReturnTwoDimes}]

set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
```